

Esse é o `init.sql` — o script que monta a estrutura do seu banco de dados. Vamos linha por linha:

-- Criação do banco de dados

Isso é um **comentário**. Tudo que vem depois de `--` em SQL é ignorado pelo banco — serve só para você (ou outra pessoa lendo o código) entender o que cada parte faz.

CREATE DATABASE IF NOT EXISTS aulaphp

Cria um **banco de dados** chamado `aulaphp`. Pense no banco de dados como uma "gaveta" onde ficam guardadas todas as tabelas do seu projeto.

O `IF NOT EXISTS` é uma proteção: se o banco `aulaphp` já existir, o comando não dá erro — ele simplesmente ignora e segue em frente. Sem isso, rodar o script duas vezes acidentalmente causaria um erro.

sql

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_general_ci;
```

Essas duas linhas definem:

- **CHARACTER SET utf8mb4**: qual "alfabeto" de caracteres o banco aceita. `utf8mb4` é o mais completo — aceita acentos (á, ç, ã), emojis, e caracteres de qualquer idioma. É o padrão recomendado hoje em dia.
 - **COLLATE utf8mb4_general_ci**: as regras de **comparação e ordenação** de texto. O `ci` significa "case insensitive" — ou seja, o banco trata "Maria" e "maria" como iguais em buscas e comparações.
-

USE aulaphp;

Diz ao MySQL: "**a partir de agora, todos os comandos seguintes são dentro do banco `aulaphp`**". É como abrir a gaveta antes de mexer no que tem dentro dela — sem isso, o comando de criar tabela não saberia em qual banco criar.

-- Criação da tabela cliente

Outro comentário, explicando a seção seguinte.

CREATE TABLE IF NOT EXISTS cliente (

Cria uma **tabela** chamada `cliente` dentro do banco `aulaphp`. Uma tabela é como uma planilha: tem colunas (os tipos de informação) e linhas (cada registro, ex: cada aluno cadastrado). O `IF NOT EXISTS` funciona igual ao de antes — evita erro se a tabela já existir.

Agora, cada linha dentro dos parênteses define uma **coluna**:

sql

IdCliente INT AUTO_INCREMENT PRIMARY KEY,

- **IdCliente**: nome da coluna
- **INT**: tipo de dado — número inteiro
- **AUTO_INCREMENT**: o banco gera esse número automaticamente, aumentando de 1 em 1 (1, 2, 3...) a cada novo registro — você nunca precisa informar esse valor manualmente
- **PRIMARY KEY**: marca essa coluna como o **identificador único** de cada linha da tabela — não pode haver dois registros com o mesmo IdCliente

sql

NomeCliente VARCHAR(50) NOT NULL,

- **VARCHAR(50)**: texto de tamanho variável, com **limite de 50 caracteres**
- **NOT NULL**: esse campo é **obrigatório** — o banco recusa o registro se vier vazio

sql

SobrenomeCliente VARCHAR(50) NOT NULL,

Mesma ideia: texto, até 50 caracteres, obrigatório.

sql

Sexo ENUM('M', 'F', 'N') NOT NULL

- **ENUM('M', 'F', 'N')**: esse campo só aceita **um desses três valores exatos** — qualquer outra coisa gera erro (foi exatamente esse tipo de erro que resolvemos antes, quando o formulário enviava um valor que não batia com o ENUM)
- Repare que **não tem vírgula no final** dessa última linha — porque é a última coluna antes de fechar o parênteses. Colocar vírgula aqui é o erro de sintaxe que corrigimos no início.

sql

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

- **ENGINE=InnoDB**: define o "motor" que vai gerenciar essa tabela por trás dos panos. InnoDB é o padrão mais usado hoje — suporta recursos importantes como relacionamentos entre tabelas (chaves estrangeiras) e transações seguras.
- **DEFAULT CHARSET=utf8mb4**: reforça, no nível da tabela, o mesmo suporte a acentos e caracteres especiais definido lá no banco.

-- Garante privilégios do usuário da aplicação sobre o banco

Comentário explicando a próxima seção.

```
GRANT ALL PRIVILEGES ON aulaphp.* TO 'aluno_user'@'%';
```

Dá **permissão total** (ALL PRIVILEGES) para o usuário aluno_user fazer qualquer operação (SELECT, INSERT, UPDATE, DELETE, etc.) em **todas as tabelas** do banco aulaphp (o .* significa "todas as tabelas desse banco").

O @'%' define **de onde** esse usuário pode se conectar — o % é um coringa que significa "de qualquer endereço/host". Isso é importante porque, no seu caso, quem está se conectando é o container web, que tem um endereço de rede interno diferente a cada vez que sobe.

```
FLUSH PRIVILEGES;
```

Diz ao MySQL para **recarregar as permissões** imediatamente, garantindo que a alteração de GRANT tenha efeito na hora, sem precisar reiniciar o servidor.

Resumindo: esse script cria a "gaveta" (aulaphp), cria a "planilha" dentro dela (cliente) com 4 colunas (id automático, nome, sobrenome e sexo restrito a 3 opções), e libera o usuário aluno_user para mexer livremente nesse banco — que é exatamente o usuário que seu cadastrar.php e consulta.php usam para conectar.

Ficou claro? Se quiser, posso explicar a diferença entre VARCHAR, INT e outros tipos de dado comuns em SQL, ou como funcionam chaves estrangeiras (FOREIGN KEY), caso seu projeto cresça para ter mais de uma tabela relacionada.